HDOC Day-62

Merging Two Sorted Arrays Using Functions, Pointers, and the Two-Pointer Technique in C

Aim

To write a C program using variable-length integer arrays, user-defined functions, pointers, and the two-pointer technique to merge two sorted arrays into a third array while maintaining ascending order and preserving duplicates.

Algorithm

1. Start the program and read the sizes of the two sorted arrays.
2. Validate that both sizes are positive.
3. Declare variable-length arrays arr1[size1] and arr2[size2].
4. Read the elements of each array in ascending order.
5. Pass the arrays and sizes to merge_sorted_arrays() using pointers.
6. Create merged[size1 + size2] and initialize indices i, j, and k to zero.
7. While both arrays contain unprocessed elements, compare arr1[i] and arr2[j].
8. Copy the smaller element to merged[k], increment its source index, and increment k.
9. Copy any remaining elements from either array.
10. Display merged[] in ascending order, separated by spaces, and stop.

C Program

```
#include <stdio.h>

void merge_sorted_arrays(int *arr1, int size1,
                         int *arr2, int size2);

int main()
{
    int size1, size2, i;

    printf("Enter the size of first array: ");
    scanf("%d", &size1);
    printf("Enter the size of second array: ");
    scanf("%d", &size2);

    if (size1 <= 0 || size2 <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr1[size1], arr2[size2];

    printf("Enter %d elements of first array "
           "in ascending order: ", size1);
    for (i = 0; i < size1; i++)
        scanf("%d", &arr1[i]);

    printf("Enter %d elements of second array "
           "in ascending order: ", size2);
    for (i = 0; i < size2; i++)
        scanf("%d", &arr2[i]);

    merge_sorted_arrays(arr1, size1, arr2, size2);
    return 0;
}

/* Function to merge two sorted arrays */
void merge_sorted_arrays(int *arr1, int size1,
                         int *arr2, int size2)
{
    int i = 0, j = 0, k = 0;
    int merged[size1 + size2];

    while (i < size1 && j < size2)
    {
        if (*(arr1 + i) <= *(arr2 + j))
        {
            merged[k] = *(arr1 + i);
            i++;
        }
        else
        {
            merged[k] = *(arr2 + j);
            j++;
        }
        k++;
    }

    while (i < size1)
    {
        merged[k] = *(arr1 + i);
        i++;
        k++;
    }
```

```
    while (j < size2)
    {
        merged[k] = *(arr2 + j);
        j++;
        k++;
    }

    printf("\nMerged Array: ");
    for (i = 0; i < size1 + size2; i++)
    {
        printf("%d", merged[i]);
        if (i < size1 + size2 - 1)
            printf(" ");
    }
    printf("\n");
}
```

Explanation

The function **merge_sorted_arrays()** receives pointers to the first elements of both arrays. Pointer expressions ***(arr1 + i)** and ***(arr2 + j)** access the elements at indices i and j. The indices i and j traverse the input arrays, while k tracks the next position in the merged array.

For example, arr1 = 1 3 5 7 and arr2 = 2 4 6 8 produce merged = 1 2 3 4 5 6 7 8. Duplicate values are retained. The algorithm runs in O(n + m) time and uses O(n + m) additional space for the merged array.

Test Case Table

Case	First Array	Second Array	Expected Merged Array
1	1 3 5 7	2 4 6 8	1 2 3 4 5 6 7 8
2	1 2 3	4 5 6	1 2 3 4 5 6
3	1 2 2 5	2 3 5 5	1 2 2 2 3 5 5 5
4	-5 -2 0	-3 1 4	-5 -3 -2 0 1 4
5	10 20 30	5 15 25	5 10 15 20 25 30
6	1	2	1 2
7	5 5 5	5 5	5 5 5 5 5
8	1 4 7	2 3 5 6	1 2 3 4 5 6 7
9	size1 = 0	1 2 3	Invalid array size
10	1 2 3	size2 = 0	Invalid array size

Sample Outputs

Test Case 1: Two sorted arrays

```
Enter the size of first array: 4
Enter the size of second array: 4
Enter 4 elements of first array in ascending order: 1 3 5 7
Enter 4 elements of second array in ascending order: 2 4 6 8

Merged Array: 1 2 3 4 5 6 7 8
```

Test Case 2: Duplicate elements

```
Enter the size of first array: 4
Enter the size of second array: 4
Enter 4 elements of first array in ascending order: 1 2 2 5
Enter 4 elements of second array in ascending order: 2 3 5 5

Merged Array: 1 2 2 2 3 5 5 5
```

Test Case 3: Negative elements

```
Enter the size of first array: 3
Enter the size of second array: 3
Enter 3 elements of first array in ascending order: -5 -2 0
Enter 3 elements of second array in ascending order: -3 1 4

Merged Array: -5 -3 -2 0 1 4
```

Test Case 4: Different sizes

```
Enter the size of first array: 3
Enter the size of second array: 4
Enter 3 elements of first array in ascending order: 1 4 7
Enter 4 elements of second array in ascending order: 2 3 5 6

Merged Array: 1 2 3 4 5 6 7
```

Test Case 5: Invalid size

```
Enter the size of first array: 0
Enter the size of second array: 3
Invalid array size!
```

Result

The C program was designed using variable-length integer arrays, pointers, user-defined functions, loops, conditional statements, and the two-pointer technique. The merge_sorted_arrays() function merges two sorted arrays into a third array in ascending order, preserves duplicates, and displays the elements separated by spaces.